

Practical Problems: Conditionals

Learning Outcome Addressed

- Recognize and use basic data operators on Booleans.
- Interpret and write the correct syntax for conditionals.
- Recognize, interpret, and write programs with conditionals.
- Recognize and interpret programs with multiple interacting functions.
- Create programs to solve simple problems.
- Identify and solve programming errors through established debugging strategies.

This assignment allows you to apply what you have learned in this module about functions. Follow the instructions below to complete problems one to eight. If you get stuck on a problem without making any progress, please reach out to the learning facilitator for assistance.

1. [15 pts] average_and_show_work

Write a function **average_and_show_work(x, y, z)** that takes three integers, averages the integers (rounding to two significant digits) and prints a string showing the work of averaging the numbers as well as the result. The function returns None (results are printed, not returned).

For example, `average_and_show_work(5, 7, 11)` would print the string `"(5 + 7 + 11) / 3 = 23 / 3 = 7.67"`, because $5 + 7 + 11 = 23$, and $23 / 3$ is $7.66666\dots$, which rounds to 7.67.

Hint 1: try putting together the three parts of the work (the initial equation, equation with the numerator added together, and final result) in three separate variables, then use a single print statement to print all three variables combined by equal signs. You may want to use the `str` built-in function.

Hint 2: to round a number to two digits after the decimal place, use the built-in `round` function's keyword argument! If you provide a second argument to `round`, it represents the number of significant digits to use.

2. [15 pts] random_gcd

Write a function **random_gcd()** that takes no parameters, randomly generates two integers between `[1, 100]`, prints those integers, then returns the greatest

common denominator (GCD) of those two integers. For example, calling `random_gcd()` might randomly generate the integers 63 and 30; the GCD of these numbers is 3 (because $63 = 3 \cdot 21$ and $30 = 3 \cdot 10$), so the function prints "x y: 63 30", then returns 3.

You should use libraries to solve this problem! Specifically, the `random` library has a useful function `randint`, and the `math` library has a useful function `gcd`; you should use both of these.

Note: the provided test cases require that you manually check whether the result was calculated correctly. The Codio test cases will check this automatically by looking at the printed numbers.

3. [20 pts] rsa

RSA (Rivest–Shamir–Adleman) is one of the most popular algorithms used to encrypt communication on the internet. It works by using a fascinating mathematical principle: if you take two prime numbers (p and q) and multiply them together (producing n), you can derive two new numbers (e and d) from p and q such that, given a number $m < n$:

$$(m^e)^d$$

mod $n = m$

Why is this useful? First, we can represent a text message as a large number (m). If we separate the two exponents, we can encrypt a message using a single number (e), transmit it over the internet, then decrypt the message using the second number (d). This lets us implement asymmetric encryption so that two people who have never shared information before can securely communicate on the internet!

$$m^e$$

mod $n = c$

$$c^d$$

mod $n = m$

Your task is to implement three functions:

The first, **encode(m, e, n)**, takes three integers (a message, the key e, and the key n) and returns the result of encoding the message with (e, n):

$$m^e$$

mod n.

The second, **decode(m, d, n)**, takes three integers (a message, the key d, and the key n) and returns the result of decoding a message with (d, n):

$$m^d$$

mod n.

The third, **transmit(m, e, d, n)**, takes four integers (a message, the key e, the key d, and the key n), calls encode to encode the message, prints the transmitted message, calls decode to decode the transmitted message, and returns the result. If it works properly, the result should be the same as the original message!

For example, we can generate the keyset (e=7, d=23, n=697) from the prime numbers 17 and 41. If we wanted to encrypt the message 402:

encode(402, 7, 697) would return 326 (because

$$402^7$$

mod 697 = 326)

decode(326, 23, 697) would return 402 (because

$$326^{23}$$

mod 697 = 402)

transmit(402, 7, 23, 697) would print "Transmitting: 326", then return 402.

Note: the math may seem intimidating, but don't worry if you don't fully understand how it works; if you can translate the given algorithms to Python, you can solve the problem!

4. [15 pts] `check_conditions`

Write a function **`check_conditions(x, y, z)`** that takes three integers and returns True if *at least one* of two conditions is met, and False otherwise. The first condition is that x and y must both be greater than 10. The second condition is that at least one of y or z is odd.

Here are a few examples to clarify:

`check_conditions(12, 14, 7)` would return True because the first condition is met (12 and 14 are both greater than 10) and the second condition is met (7 is not even).

`check_conditions(15, 1, 9)` would return True because the second condition is met (both 1 and 9 are not even) even though the first condition is not (15 is greater than 10, but 1 is not).

`check_conditions(10, 12, -2)` would return False because neither condition is met (10 is not greater than 10, and 12 and -2 are both even).

Note: you could solve this using conditionals, but you don't need to! Try solving it with just logical and Boolean operations instead.

5. [15 pts] `estimate_shipping_time`

Write the function **`estimate_shipping_time(month, day)`** that takes two parameters, a string (the month) and integer (the day), and returns an integer, the estimated number of days it will take to ship an order from a fictional company. Normally orders take 10 days to ship; however, there are a few cases where the estimated number of days changes...

The month of December is especially busy. Orders placed in December take 5 extra days to ship.

Shipments are particularly delayed at the end of December (from the 25th onwards) due to holidays; orders placed between then and the beginning of January then take another 5 days to ship (twenty days total).

In any month other than December, orders ship 3 days faster in the first seven days of the month, because new stock has arrived.

For example, `estimate_shipping_time("April", 4)` would return 7 because the shipping date is not in December and occurs in the first seven days of the month. On the other hand, `estimate_shipping_time("December", 24)` would return 15 because the order is placed in December, but before the holiday period.

Hint: check the test cases if you're confused about what the exact boundaries should be!

6. [20 pts] Fix the Code

You've been given a function **`is_nice_outside`**, which takes three parameters (the temperature as an integer, whether or not the temperature is represented in Fahrenheit as a Boolean, and whether or not it is raining as a Boolean) and returns True if it is nice outside and False otherwise. To determine whether or not it is nice outside, you check the following conditions:

If it is raining, it is not nice outside.

If it's not raining and the temperature is greater than 4 degree C and less than 35 degree C, it is nice outside!

If the temperature is not in that range, it's not nice outside.

The temperature might sometimes be given in Fahrenheit instead of Celsius, so there is also a helper function **`fahrenheit_to_celsius`** which takes the temperature as an integer and returns the temperature represented in Celsius as a float.

The code currently has several bugs that need to be fixed. Use debugging approaches to find and fix the bugs so that the code runs properly and passes the test cases.

Hint: The helper function may also have some bugs!

7. [20 pts] Get_grade

Write a function **get_grade** that takes five numbers representing quiz grades and an optional keyword argument, **curve** (a number that represents an overall curve), and returns the average quiz grade with the lowest grade dropped and the curve applied. Note that applying a curve means adding the amount of curve points to all the quiz grades. The function also prints the average grade pre-curve (but post dropping the lowest score). For example, `get_grade(75, 80, 85, 90, 95, curve=2)` would print “Average grade pre-curve: 87.5” and return 89.5 because $((75 + 80 + 85 + 90 + 95) - 75) / 4 = 87.5$, and $87.5 + 2 = 89.5$ (after applying the curve).

Keep in mind that **curve** is an optional keyword argument - that means the function still has to work if it isn't provided! For example, `get_grade(82, 93, 87, 64, 91)` would print “Average grade pre-curve: 88.25” and return 88.25 because $((82 + 93 + 87 + 64 + 91) - 64) / 4 = 88.25$, and the average is not changed when no curve is applied. In other words, the curve has a default value of 0 unless otherwise specified.

For this problem, we have not provided a function header. Set up the function yourself so that it works with the provided test cases.

8. [30 pts] Identify_dog_breed

Write the function `identify_dog_breed` which takes two characteristics of a dog - their weight (an integer) and their coat length (a string, short, medium, or long) and returns a string, the name of the dog species described by the characteristics. To simplify the problem, we will only consider dogs in the Herding category, and only a subset of the dogs. Your program should correctly identify the following dogs:

Weight	Coat Length	Breed
[20, 50) lbs	medium	Australian Shepherd

Weight	Coat Length	Breed
[20, 50) lbs	long	Bearded Collie
>= 80 lbs	short	Beauceron
[50, 80) lbs	short	Belgian Malinois
>= 80 lbs	medium	Bouvier des Flandres
[50, 80) lbs	long	Collie
[50, 80) lbs	medium	German Shepherd
< 20 lbs	medium	Mudi
>= 80 lbs	long	Old English Sheepdog
[20, 50) lbs	short	Pembroke Welsh Corgi
< 20 lbs	long	Shetland Sheepdog
< 20 lbs	short	Swedish Vallhund

For example, `identify_dog_breed(25, "short")` would return "Pembroke Welsh Corgi", while `identify_dog_breed(95, "long")` would return "Old English Sheepdog".

Hint: You'll want to use conditionals to solve this problem. In particular, you should consider using nested conditionals to solve it more easily!

Write the Algorithm for the above Problems here.

